UNIT 5 · TOPIC 5.5

Protecting Applications

Quiz

Instructions: This quiz has 9 multiple-choice questions, 1 point each. Circle the letter of the single best answer for each question.

Name: **Date:** **Class:**

1. Which of the following BEST describes the application security principle of "secure by design"?

- ○ A. Security controls are applied to an application immediately before it is released to production.
- ○ B. Security is incorporated into the application from the earliest stages of planning and architecture.
- ○ C. All application data is encrypted at rest after the product is deployed.
- ○ D. Developers conduct penetration testing once a vulnerability is publicly reported.

2. A software company releases a new application in which all user accounts are created with minimal permissions by default, and stronger access must be explicitly granted by an administrator. Which application security principle does this BEST illustrate?

- ○ A. Secure by design
- ○ B. Defense in depth
- ○ C. Security by default
- ○ D. Vulnerability management

3. Which of the following development activities is MOST consistent with the secure by design principle?

- ○ A. Running an automated vulnerability scanner against the completed application one week before launch
- ○ B. Issuing a security patch after users report unusual application behavior
- ○ C. Including threat modeling as a required step during the application planning phase
- ○ D. Disabling security logging during development to improve build speed

4. A web application allows users to enter a username in a search field. The application passes the raw user input directly into a database query without modification. Which attack does the absence of input sanitization make MOST likely, and why?

- ○ A. A SQL injection attack, because unsanitized input can include SQL syntax that alters or exposes data in the backend database.
- ○ B. A brute-force attack, because the application cannot limit the rate of queries submitted by a single user.
- ○ C. A man-in-the-middle attack, because unencrypted input can be intercepted as it travels between the browser and server.
- ○ D. A denial-of-service attack, because unfiltered input can exhaust server memory by creating infinite loops in the query engine.

5. A developer configures a comment field on a blog to accept only plain text, stripping angle brackets, script tags, and all HTML before storing the input. How does this sanitization control protect other users of the application?

- ○ A. It protects data in transit by encoding comments before they are sent over the network.
- ○ B. It prevents attackers from discovering other users' session tokens by limiting the length of stored comments.
- ○ C. It blocks stored cross-site scripting (XSS) attacks by ensuring malicious script tags cannot be embedded in content that other browsers will render.

○ D. It prevents SQL injection by removing characters that could modify a database query.

6. A developer rewrites all database queries in the application to use parameterized statements (prepared statements) instead of string concatenation. How does this change protect the application from injection attacks?

○ A. The database encrypts each query before execution, making it unreadable to attackers monitoring network traffic.

○ B. The application rate-limits the number of queries any single user can submit per minute, slowing down automated attacks.

○ C. The database rejects any query that contains more than one SQL statement, blocking multi-statement injection payloads.

○ D. The query structure is compiled separately from the user-supplied data, so user input can never be interpreted as executable SQL code.

7. A security review of an e-commerce application reveals that product review fields render raw HTML entered by users directly in other customers' browsers. The development lead argues the issue should be addressed in a future release to avoid delaying the current launch. Evaluate this decision from the perspective of application security principles.

○ A. The decision is acceptable because stored XSS is a low-severity issue that primarily affects only the attacker's own browser session.

○ B. The decision is problematic because it exposes every user who views a poisoned review to malicious script execution, and it reflects a failure of both secure by design and security by default principles.

○ C. The decision is acceptable because HTTPS encryption will prevent injected scripts embedded in page content from executing in users' browsers.

○ D. The decision is problematic only if the application stores payment card data, since PCI-DSS would mandate an immediate fix.

8. Two development teams are building separate patient data portals. Team Alpha conducts threat modeling and embeds input validation controls during the architecture phase. Team Beta builds the full UI first and then adds input validation and access controls in the final testing sprint. Both applications pass the same pre-launch security scan with zero critical findings. Which evaluation of the two approaches is MOST accurate from an application security standpoint?

○ A. Team Alpha's approach is more sound because security controls architected from the start are less likely to contain design-level flaws that automated scanners cannot detect, and remediation costs are far lower when issues are caught during design.

○ B. Team Alpha's approach is preferable only in regulated industries; for general applications the approaches carry equivalent risk.

○ C. Team Beta's approach is more efficient because it avoids slowing down the development process with security considerations that can be validated at the end.

○ D. Both approaches are equally secure because they produced the same scan results, and the timing of security activities does not affect the final security posture.

9. A penetration tester discovers that a file-management application accepts a target file path directly from user input and passes it to a server-side file-read operation with no sanitization or path restriction. The application owner argues that because all users must be authenticated, the risk is low and remediation can be deferred. Evaluate this reasoning and identify the correct remediation approach.

○ A. The reasoning is sound; authentication ensures that only authorized users access the system, which sufficiently limits the impact of unsanitized file paths.

○ B. The reasoning is flawed, but the correct remediation is to replace the file system with a database, eliminating direct file-path handling entirely.

○ C. The reasoning is sound; path traversal is only exploitable over unauthenticated connections, and the finding should be marked as informational rather than a vulnerability.

○ D. The reasoning is flawed; authentication does not prevent an authenticated user from supplying traversal sequences such as ../../etc/shadow to read files outside the intended directory, and remediation requires canonicalizing and allowlist-validating the resolved path before any file operation.